White Paper

On-chain Agent Identity, Authority, Account Opening, and Payment Base Layer

A technical blueprint for launching an agent-first financial identity stack

Prepared for	An agent-finance architecture that starts with identity, then authority, then accounts and payments.
Scope	Reference architecture, threat model, implementation choices, sample data models, phased roadmap, and source references.
Design stance	Keep PII and raw KYC data off-chain; put identifiers, credential hashes, status, and execution attestations on-chain; use verifiable credentials and policy engines off-chain.

Key thesis: in an agent-native financial market, identity and delegated authority are upstream of KYC reuse, wallet creation, payment, trading, settlement, and audit. Build the identity and authority layer first.

1. Executive summary

The recommended build path is to start with an identity-and-authority base layer for on-chain agents, not with a full exchange or a consumer wallet. The minimum coherent stack is: Agent DID, owner-to-agent linkage, verifiable credentials for ownership/KYC/authority, smart accounts for agent wallets, a policy engine for delegated spend and trade permissions, and a payment-intent execution layer that can enforce human-in-the-loop or pre-authorized automation.

- Do not put raw PII, passports, company filings, or full KYC payloads on-chain.
- Do put identifiers, credential hashes, revocation status, controller changes, policy commitments, wallet bindings, and execution attestations on-chain when they improve verifiability.
- Use W3C DID + VC as the identity and claims model; use OpenID4VCI/OpenID4VP or a similar presentation flow for issuance and presentation; use smart accounts, preferably ERC-4337 style, for programmable agent wallets.
- Start with payments before trading. Agent payment rights, budget limits, and auditability are easier to validate than autonomous market access.
- Model three execution modes from day one: human-confirmed, pre-authorized automation, and fully agent-native operation under strict policies.

2. Reasoning summary (concise, non-private)

You asked to see the reasoning process. I cannot provide hidden private chain-of-thought in raw form, but I can provide the decision summary that guided this design. The architecture below is based on four design choices:

1. Identity comes before finance. If an agent cannot be uniquely identified, linked to an owner, issued permissions, and revoked, then KYC reuse, account opening, payment, and autonomous trading cannot be made safe or auditable.
2. DID alone is not enough. DID identifies a subject and points to control material and services. KYC results, ownership claims, and spend/trade permissions belong in verifiable credentials and policy rules, not directly in a DID document.
3. The chain should anchor trust, not store all trust data. On-chain state should be small, durable, and privacy-preserving: identifiers, hashes, revocation status, policy commitments, and receipts. Off-chain systems should store sensitive and mutable business data.
4. Programmable accounts are required. An agent-finance stack needs smart accounts or a comparable policy-enforced account model so that budgets, rate limits, counterparties, and kill switches can be enforced at the account layer rather than by application convention alone.

3. Existing references and what they prove

No single standard today fully solves on-chain agent identity, KYC-backed account opening, and autonomous payment in one package. The useful starting point is to combine existing standards and reference implementations.

Layer	Purpose	What lives here	Notes
DID	Decentralized identifier	Identifier syntax, DID documents, controller, verification methods, services	General identity primitive; not KYC-specific
VC	Claims and credentials	Ownership, KYC outcome, policy, authority credentials	Right place for agent ownership and permissions
OID4VCI / OID4VP	Credential issuance and presentation	Wallet-to-issuer and wallet-to-verifier flows	Useful for account opening and presenting KYC/authority
ERC-4337 smart accounts	Programmable wallet/account	Policy logic, paymasters, session keys, user operations	Strong fit for agent wallets
AgentKit / agentic wallets	Operational reference	Shows agents can safely hold wallets and act on-chain	Useful implementation reference, not identity standard
x402 / AP2	Payment and	Machine-payments,	Useful for payments and

/ ERC-8183	agent commerce references	delegated payment, escrowed jobs	later agent-to-agent commerce

3.1 What the main references contribute

- W3C DID defines a verifiable decentralized identifier and DID Document model for any subject, not only people. It supports controllers, verification methods, and service endpoints.
- W3C VC defines a tamper-evident claims model with issuer, holder, and verifier roles. This is the correct place to express ownership, KYC status, and authority claims.
- OpenID4VCI and OpenID4VP define interoperable issuance and presentation flows for credentials, which are practical building blocks for onboarding and account opening.
- ERC-4337 smart accounts allow arbitrary verification logic and policy-driven account behavior, which is essential for agent wallets with delegated authority, session keys, or paymasters.
- Coinbase AgentKit and agentic wallet initiatives demonstrate that AI agents can hold wallets and perform on-chain actions, but they do not replace a full identity and authority model.
- Google AP2, x402, and ERC-8183 are useful for delegated payments, machine-native payment flows, and escrowed agentic commerce, but they should sit on top of a stronger identity base layer.

4. Problem statement and design goals

4.1 Problem statement

The core problem is not simply “how do we let an AI agent send tokens?” The real problem is how to make an on-chain agent a recognizable, governable, auditable, revocable financial actor. The stack must support all of the following without leaking sensitive identity data:

- Prove that an agent exists and can be resolved to control material and service endpoints.
- Prove who owns or sponsors the agent.
- Prove that some KYC/KYB or equivalent compliance process has been completed for the right real-world party.
- Bind those claims to a wallet or smart account that can hold assets or interact with venues.
- Constrain the agent’s financial powers through explicit policy.
- Record enough immutable state to support post-trade audit and revocation.

4.2 Design goals

- Identity-first, not wallet-first.
- Privacy-preserving and minimization-first.
- Compatible with human-in-the-loop, pre-authorized, and autonomous operation.
- Portable across chains and venues where possible.
- Amenable to later extensions: venue access, trading authority, collateral, and settlement receipts.

4.3 Non-goals for v1

- Do not build a universal KYC provider in v1.
- Do not put raw documents on-chain.
- Do not solve every DID method or wallet format in v1.
- Do not begin with fully autonomous trading across all venues; start with payments and narrow permissions.

5. Reference architecture

Layer	Purpose	What lives here	Notes
A	Identity plane	Owner DID, Agent DID, DID Documents, controller model, service endpoints	Creates the resolvable identity graph
B	Credential plane	Ownership VC, KYC/KYB VC, Authority VC, Risk VC	Carries claims and permissions
C	Account plane	Agent smart account, treasury sub-account, key manager	Holds and moves assets
D	Policy plane	Spend rules, counterparty allowlists, asset/venue limits, expiry, approvals	Enforces financial authority
E	Execution plane	Payment intent API, quote, authorization, settlement adapter	Turns policy into action
F	Audit plane	On-chain attestation, receipt hash, revocation registry, event logs	Supports forensics, compliance, and dispute handling

5.1 Recommended subject model

- Owner: a natural person, organization, fund, platform, or treasury that bears legal or economic responsibility.
- Agent: an autonomous or semi-autonomous software actor with a unique DID and a wallet or account binding.
- Issuer: a trusted identity, KYC, or policy issuer able to sign credentials.
- Verifier: a wallet service, venue, payment service, or policy engine that checks credentials and policy before execution.
- Policy authority: the entity that can grant, rotate, suspend, or revoke the agent's financial rights.

5.2 Minimal trust graph

```
Owner DID  ----(owns/sponsors)--->  Agent DID
Agent DID  ----(bound to)------->  Smart Account
Issuer DID ----(issues VC)------->  Ownership VC / KYC VC / Authority VC
Verifier ----(checks)------------>  DID resolution + VC validity + policy + revocation
Execution ----(writes)----------->  On-chain payment or attestation receipt
```

6. Identity layer in detail

6.1 DID method strategy

There are three realistic choices for v1.0, each with trade-offs:

Layer	Purpose	What lives here	Notes
did:key	Fastest to start	No registry required; simple self-certifying identifier	Poor fit for mutable lifecycle, rotation history, and rich service discovery
did:web	Operationally simple	Easy control through standard web infrastructure; human-readable	Depends on domain control; less natively on-chain
Custom method (e.g., did:agent)	Best long-term fit	Can define lifecycle, revocation, canonical bindings, and chain anchoring explicitly	Highest design and implementation burden

Recommendation: start with a hybrid model. Use a web-backed issuer and verifier trust domain for speed, but bind agent state changes and wallet linkage to on-chain events. If the system succeeds, define a custom DID method later for richer lifecycle semantics.

6.2 DID document fields that matter most for agents

- id: the canonical DID of the agent.
- controller: who can rotate keys or alter the DID document; this should usually be a policy or owner authority, not the hot transaction key.
- verificationMethod: public keys or JWK references used for authentication, assertion, or capability work.
- authentication: methods the agent can use to prove control.
- assertionMethod: methods used to sign claims or execution attestations.
- capabilityInvocation and capabilityDelegation: especially relevant for delegated authority and later capability-based security models.
- service: service endpoints such as policy endpoint, presentation endpoint, messaging endpoint, or account-opening endpoint.

```
{
  "id": "did:web:agents.example.com:agent:8f1d",
  "controller": ["did:web:treasury.example.com"],
  "verificationMethod": [{
    "id": "did:web:agents.example.com:agent:8f1d#auth-1",
    "type": "JsonWebKey",
    "controller": "did:web:agents.example.com:agent:8f1d",
    "publicKeyJwk": {"kty": "EC", "crv": "P-256", "x": "...", "y": "..."}
  }],
  "authentication": ["did:web:agents.example.com:agent:8f1d#auth-1"],
  "assertionMethod": ["did:web:agents.example.com:agent:8f1d#auth-1"],
  "service": [{
    "id": "did:web:agents.example.com:agent:8f1d#policy",
    "type": "AgentPolicyService",
    "serviceEndpoint": "https://policy.example.com/agents/8f1d"
  }]
}
```

7. Credential layer in detail

A DID identifies. A credential says something about that identified subject. For this design, credentials should be issued off-chain, held by a wallet or identity agent, presented selectively, and anchored on-chain only when necessary.

7.1 Recommended credential set

Layer	Purpose	What lives here	Notes
Ownership VC	Binds an agent DID to an owner DID or organization DID	Who is ultimately responsible for the agent	
KYC/KYB Result VC	States that required verification was completed by a trusted issuer	Whether the owner or institution passed onboarding	
Authority VC	States what the agent may do financially	Spend, payment, venue, asset, or settlement permissions	
Risk or Status VC	States operational status or risk level	Suspended, probationary, reduced limit, or region restricted	

7.2 Suggested authority VC claims

```
{
  "type": ["VerifiableCredential", "AgentAuthorityCredential"],
```

```
  "issuer": "did:web:issuer.example.com",
  "credentialSubject": {
    "id": "did:web:agents.example.com:agent:8f1d",
    "owner": "did:web:fund.example.com",
    "allowedActions": ["payment.execute", "payment.intent.create"],
    "allowedAssets": ["USDC", "USDT"],
    "counterpartyPolicy": {"allowList": ["merchant:alpha", "api:beta"]},
    "amountLimits": {"single": "100.00", "daily": "1000.00"},
    "regions": ["SG", "UAE"],
    "requiresHumanApprovalAbove": "25.00",
    "validUntil": "2026-12-31T23:59:59Z"
  }
}
```

7.3 Issuance and presentation protocols

- Use OpenID4VCI or an equivalent OAuth-style issuance flow to issue credentials to the owner wallet or custody layer.
- Use OpenID4VP or a comparable presentation protocol when a wallet, venue, or payment verifier needs proof of KYC or authority.
- Support selective disclosure where possible so verifiers learn “this agent is authorized” without receiving full KYC payloads.
- Maintain a credential status and revocation registry that can be checked during payment or account opening.

8. Account opening and wallet architecture

8.1 What “account opening” means for agents

In this stack, account opening is not a form-submission event. It is a state transition from “identified and authorized subject” to “subject with a wallet or account that can act within policy.”

- Create or assign an agent smart account.
- Bind the account to the Agent DID.
- Attach policy configuration and approved signers/session keys.
- Record the account-opening event and account status in an auditable registry.
- If needed, bind the agent account to a custodian, exchange sub-account, or internal treasury ledger.

8.2 Recommended wallet stack

- Use ERC-4337 style smart accounts on an EVM chain or L2 for programmable verification logic, paymasters, and account-level policy support.
- Use a separate key hierarchy: cold governance keys for recovery and rotation; service keys for identity operations; short-lived session keys for low-risk execution.
- Support paymasters only after the identity and policy checks pass, to avoid subsidizing abusive traffic.

- For enterprises, add sub-accounts or treasury partitions so multiple agents can draw from bounded budgets rather than a shared hot wallet.

8.3 Recommended on-chain account registry fields

```
struct AgentAccountBinding {
    bytes32 agentDidHash;
    address smartAccount;
    bytes32 currentAuthorityCredentialHash;
    uint64 status; // active, suspended, revoked
    uint64 createdAt;
    uint64 updatedAt;
}
```

9. Authorization and policy engine

The policy engine is the real safety boundary. It decides whether a requested action is allowed now, by this agent, against this counterparty, with this asset, at this size, under this execution mode.

9.1 Core policy dimensions

- Action scope: payment creation, payment execution, refund request, venue order submission, or custody withdrawal.
- Asset scope: which tokens or currencies are permitted.
- Counterparty scope: allowlist, denylist, jurisdiction restrictions, or trust/risk thresholds.
- Budget scope: single-transaction, rolling daily, rolling monthly, or strategy-lifetime caps.
- Approval scope: thresholds that trigger human approval or step-up authentication.
- Time scope: expiry windows, market hours, cooling-off windows, or kill-switch states.

9.2 Execution modes

Layer	Purpose	What lives here	Notes
Human confirmed	Agent proposes; owner confirms each material action	Initial production rollout, high value or high risk payments	
Pre-authorized	Agent acts automatically within narrow policy	Recurring or low-risk payments; preferred v1 automation mode	
Fully agent-native	Agent acts without immediate human confirmation but under enforceable policy and audit	Later stage, only after strong controls and monitoring	

9.3 Policy decision pseudocode

```
if agent_status != ACTIVE: reject
if credential_revoked(authority_credential): reject
if requested_asset not in allowed_assets: reject
if counterparty not allowed: reject
if amount > single_limit: reject
if daily_spend + amount > daily_limit: reject
if amount > human_approval_threshold: require_approval
execute_or_hold_intent()
```

10. Payment execution layer

A robust payment base layer should use payment intents rather than direct transfer requests. Intents preserve context, give the policy engine time to validate claims, and provide a durable audit object.

10.1 Payment intent lifecycle

1. Create intent: agent requests a payment or settlement action with context, amount, asset, counterparty, purpose, and requested execution mode.
2. Resolve and verify: verifier resolves DID, checks credential validity, checks revocation and account status.
3. Apply policy: policy engine decides allow, hold for approval, or deny.
4. Execute: wallet submits a user operation or chain transaction, or calls an off-chain settlement connector.
5. Attest and record: write receipt hash or attestation on-chain and store rich logs off-chain.

10.2 Suggested payment intent object

```
{
  "intentId": "pi_01H...",
  "agentDid": "did:web:agents.example.com:agent:8f1d",
  "account": "0xabc...",
  "counterparty": "merchant:alpha",
  "asset": "USDC",
  "amount": "12.50",
  "purpose": "api-call-settlement",
  "mode": "pre_authorized",
  "credentialRefs": ["cred_ownership", "cred_authority"],
  "policyRef": "policy_v7",
  "createdAt": "2026-03-14T10:00:00Z"
}
```

10.3 Payment connectors

- On-chain stablecoin transfer for agent-native settlement.

- Escrow release flow for condition-based completion.
- Internal ledger movement for high-frequency or micro-payment environments.
- Optional bridge to fiat rails later, but only after the authority model is stable.

11. On-chain versus off-chain data placement

A common failure mode is trying to make “identity on-chain” mean “all identity data on-chain.” That is not recommended. The better model is anchored verifiability.

Layer	Purpose	What lives here	Notes
On-chain	DID state anchor, wallet binding, credential hash, credential status, revocation status, policy commitment hash, payment or settlement attestation, account status flags	Small, durable, privacy-conscious, auditable	
Off-chain	Raw KYC files, onboarding forms, sanctions evidence, full credential payloads when sensitive, internal risk scores, detailed payment context logs	Sensitive, mutable, often legally erasable or access controlled	

11.1 Why this split works

- It supports privacy laws and operational minimization.
- It avoids putting revocable secrets or mutable business logic entirely on-chain.
- It still provides a cryptographic anchor and tamper-evident trail for what mattered at the time of execution.

12. Security, privacy, recovery, and revocation

12.1 Security controls

- Separate identity control keys from execution keys.
- Use short-lived session keys for low-risk automation and revoke them aggressively.
- Keep a governance or recovery key path for emergency account suspension.
- Require counterparty, asset, and value policy checks before paymaster sponsorship or execution.
- Log every policy evaluation result, not just successful executions.

12.2 Revocation and suspension

- Credentials must have status endpoints or revocation references that verifiers can check in real time or near real time.
- Agent accounts should support suspended and revoked states distinct from deleted.
- A revoked authority VC should cause future actions to fail even if the wallet still has funds.

12.3 Recovery

- Plan for key rotation in DID Documents.
- Plan for wallet migration from one smart account implementation to another.
- Keep owner-to-agent bindings re-issuable when a wallet changes but the agent identity remains valid.

13. Rollout plan

13.1 Phase 0 — design and trust model

- Choose chain and wallet model. Practical default: EVM L2 + ERC-4337 smart accounts.
- Choose DID approach. Practical default: web-backed issuer domain plus on-chain state anchoring.
- Define four credential schemas: ownership, KYC/KYB result, authority, status/risk.
- Define on-chain registry contracts for agent-account binding and revocation state.

13.2 Phase 1 — MVP

- Issue Agent DID and Owner DID.
- Issue Ownership VC and Authority VC after KYC/KYB is completed through an off-chain provider.
- Create ERC-4337 smart account for the agent.
- Implement payment intent API and policy engine.
- Support USDC/USDT payment execution only, with narrow allowlists and low limits.
- Write attestation receipts and registry updates on-chain.

13.3 Phase 2 — production hardening

- Add OpenID4VCI/OpenID4VP or equivalent interoperable issuance/presentation flows.

- Add revocation webhooks and verifier caching rules.
- Add internal risk scoring and adaptive policy thresholds.
- Add enterprise treasury partitions and delegated admin controls.

13.4 Phase 3 — controlled extensions

- Venue access and trading authority credentials.
- Escrowed agent-to-agent commerce flows.
- Selective fiat bridge or custodial account opening adapters.
- Proof-of-service and later agent-to-agent market infrastructure.

14. Build versus buy

Layer	Purpose	What lives here	Notes
Build	Policy engine, on-chain registry, account bindings, authority schema, audit model	These are core differentiation and safety boundaries	
Adopt	DID/VC standards, OpenID4VCI/OpenID4VP, ERC-4337 account framework	Use open standards where they are strong	
Integrate	KYC/KYB providers, custodians, sanctions providers, wallet infra	These are capability providers, not the core moat	

AP2 should be adopted as an authorization-and-liability design input, not as the full substrate. The recommended strategy is to build the identity, credential, account, and policy core in-house, then make that core AP2-compatible. In practical terms: build Agent DID, Owner/Institution DID, Ownership VC, KYC/KYB VC, Authority/Policy VC, account-opening logic, smart-account controls, revocation, and auditable on-chain receipts as first-party infrastructure; reuse AP2 concepts for mandate semantics, trusted-surface challenge handling, and payment-authorization packaging toward merchants, issuers, or credential providers. This gives the stack long-term control over agent identity and delegated authority while remaining compatible with agentic commerce protocols as they mature.

The target is AP2-compatible but not AP2-dependent. A verifier or commerce connector should be able to transform internal authority and payment-intent objects into AP2-like mandate artifacts when interacting with AP2-aware merchants, credential providers, or payment processors. Internally, however, the source of truth remains the DID/VC/account/policy stack.

14.4 Compatibility target

The platform should inherit AP2's mandate vocabulary and trust choreography. Specifically: Cart Mandate inspires a concrete user-confirmed action object; Intent Mandate inspires pre-authorized delegated authority for later execution; Payment Mandate inspires a compact authorization artifact carried into the payment ecosystem. The AP2 trusted-surface model should be adopted for step-up verification, challenge resolution, and dispute evidence.

14.3 What we inherit from AP2

The in-house core should own five primitives: (1) identity registry and DID resolution rules for owners, institutions, and agents; (2) verifiable credentials for ownership, KYC/KYB result, authority, and status/risk; (3) account-opening orchestration that maps validated credentials into agent smart accounts and treasury partitions; (4) policy evaluation and revocation services; and (5) execution receipts and audit anchoring. These are the durable control points of the system.

14.2 What we build ourselves

Use AP2 for what it is best at: explicit transaction authorization, human-present and human-not-present mandate patterns, challenge/step-up routing to a trusted surface, and payment-ecosystem visibility through mandate objects. Do not rely on AP2 to define agent identity, KYC/KYB representation, wallet/account opening, or generalized agent-to-agent financial authority.

14.1 Recommended boundary with AP2

15. What success looks like

- An agent can be resolved, authenticated, and linked to a responsible owner.
- A verifier can confirm that required KYC/KYB was completed without seeing raw documents.
- A wallet or smart account can be opened automatically after credentials are presented and validated.
- The agent can pay under bounded authority, with every action attributable, revocable, and auditable.
- The same foundation can later support venue access, trading, escrow, and agent-to-agent settlement.

16. Selected source references

The references below are the most relevant official or primary materials for building this stack. They are included as implementation references, not as endorsements of any specific vendor.

W3C DID 1.1 / DID Core family — DID syntax, DID documents, controllers, verification methods, and services. https://www.w3.org/TR/did-1.1/

W3C DID 1.0 Recommendation decision — Confirms DID Core advanced to W3C Recommendation in 2022. https://www.w3.org/2022/06/DIDRecommendationDecision.html

W3C Verifiable Credentials Data Model v2.0 — Defines issuer, holder, verifier model and tamper-evident credential data model. https://www.w3.org/TR/vc-data-model-2.0/

OpenID for Verifiable Credential Issuance 1.0 — OAuth-based issuance flow for VCs. https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html

OpenID for Verifiable Presentations 1.0 — Presentation flow for credentials to verifiers. https://openid.net/specs/openid-4-verifiable-presentations-1_0.html

EIP-4337 — Smart contract accounts, UserOperations, bundlers, and paymasters. https://eips.ethereum.org/EIPS/eip-4337

Coinbase AgentKit — Reference implementation direction for AI agents with secure wallet management and on-chain actions. https://docs.cdp.coinbase.com/agent-kit/welcome

x402 — Machine-native stablecoin payments over HTTP. https://docs.cdp.coinbase.com/x402/welcome

Google Agent Payments Protocol (AP2) — Delegated and interoperable agent-led payment design direction. https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol

ERC-8183 Agentic Commerce — Escrowed job and evaluator pattern for later agent-to-agent commerce. https://eips.ethereum.org/EIPS/eip-8183

Appendix A. Detailed technical specification

This appendix expands the main white paper into concrete implementation-level guidance. It focuses on object models, cryptographic trust boundaries, smart-contract modules, APIs, and operational flows for an agent identity, authority, account opening, and payment base layer.

A.1 System design principles

- Separate identity from execution. A DID should identify an owner or agent, while the wallet account should execute payments and on-chain actions under policy control.
- Separate claims from state. Credentials should carry KYC/KYB and authority claims, while on-chain contracts should carry compact registries, revocation status, hashes, and bindings.
- Default to least privilege. Agent rights should start narrow: limited assets, limited counterparties, low limits, and short-lived authorizations.
- Make policy checkable before gas is spent. All high-cost operations should be gated by preflight verification, then enforced again at account execution time.
- Keep PII off-chain. Only identifiers, credential references, hashes, status bits, and auditable execution receipts should be on-chain.

A.2 Recommended architecture stack

Layer	Recommended choice for v1	Why it fits	Upgrade path
Identifier layer	W3C DID with a hybrid did:web issuer domain plus chain-anchored state commitments	Fast to ship, easy verifier onboarding, clear web trust domain	Add a bespoke did:agent method or anchor to did:ion / did:pkh later
Credential layer	W3C VC 2.0 with OpenID4VCI / OpenID4VP issuance and presentation	Interoperable issuance and selective disclosure roadmap	Add BBS+ or SD-JWT VC where verifier minimization matters
Wallet layer	ERC-4337 smart accounts on an EVM L2	Programmable validation, paymasters, session keys, modular controls	Add MPC custody, cross-chain wallet abstraction, or enterprise subaccounts
Policy layer	Off-chain policy engine with on-chain policy commitments	Fast iteration for rules; on-chain commitments preserve auditability	Move critical checks fully on-chain if latency and gas allow
Payment layer	Payment intent service plus smart-account execution	Supports human-approved and autonomous modes with the same object model	Extend to settlement, escrow, and trade intents

Audit layer	On-chain receipt hashes and off-chain signed logs	Good privacy-performance trade-off	Add zk attestations or verifiable execution proofs later

A.3 DID model and lifecycle

A.3.1 Subject types

- Owner DID: the natural person, enterprise, fund, or treasury with legal/economic responsibility.
- Institution DID: the legal entity or service provider that issues credentials, opens accounts, or operates policy.
- Agent DID: the software actor that receives delegated powers and binds to a wallet account.
- Service DID (optional): a payment connector, KYC provider, custodian, or settlement service for trust graph clarity.

A.3.2 Minimal DID document fields

```
{
  "id": "did:web:agents.example.com:agent:abc123",
  "controller": ["did:web:org.example.com:treasury", "did:web:org.example.com:policy"],
  "verificationMethod": [
    {"id": "#auth-1", "type": "JsonWebKey2020", "controller":
"did:web:agents.example.com:agent:abc123", "publicKeyJwk": {"kty": "OKP", "crv": "Ed25519", "x":
"..."}},
    {"id": "#sess-1", "type": "JsonWebKey2020", "controller":
"did:web:agents.example.com:agent:abc123", "publicKeyJwk": {"kty": "EC", "crv": "secp256k1", "x":
"...", "y": "..."}}
  ],
  "authentication": ["#auth-1"],
  "assertionMethod": ["#auth-1"],
  "capabilityInvocation": ["#sess-1"],
  "service": [
    {"id": "#policy", "type": "AgentPolicyService", "serviceEndpoint":
"https://policy.example.com/v1/agents/abc123"},
    {"id": "#presentation", "type": "OpenID4VPVerifier", "serviceEndpoint":
"https://verify.example.com/vp"}
  ]
}
```

A.3.3 DID lifecycle events

- Create: owner and platform mint Agent DID, set controller set, and write an initial state commitment.
- Bind: link DID to smart account, policy profile, and credential status list references.
- Rotate: rotate auth or session keys without changing the DID.
- Suspend: freeze payment and trading rights while preserving audit state.
- Deactivate: permanently retire the DID while keeping historical references resolvable for audit.

A.4 Credential schema design

A.4.1 Core credential families

Credential	Subject	Key claims	Checked by
Ownership VC	Agent DID	ownerDid, sponsorDid, creationAuthority, revocationReference	account opener, payment verifier, venue

KYC/KYB VC	Owner DID or Institution DID	jurisdiction, kycStatus, riskTier, reviewDate, expiry	account opener, bridge, regulated venue
Authority VC	Agent DID	allowedActions, assetAllowlist, counterpartyRules, limitProfile, autonomyMode	policy engine, wallet validator
Account Binding VC	Agent DID	accountAddress, chainId, custodyModel, signerProfile	payment executor, auditor
Connector Access VC	Agent DID	connectorId, railType, regionPermissions, sandboxOrProd	connector gateway

A.4.2 Suggested Authority VC payload

```
{
  "credentialSubject": {
    "id": "did:web:agents.example.com:agent:abc123",
    "ownerDid": "did:web:org.example.com:treasury",
    "autonomyMode": "pre_authorized",
    "allowedActions": ["payment:create", "payment:execute"],
    "assetAllowlist": ["USDC", "USDT"],
    "counterpartyPolicy": {"mode": "allowlist", "listRef": "cp-list-2026-01"},
    "limits": {"singleTx": "100.00", "daily": "1000.00", "monthly": "5000.00"},
    "jurisdictionPolicy": {"deny": ["IR", "KP"], "allow": ["US", "SG", "AE"]},
    "approvalThresholds": {"humanConfirmationAbove": "25.00", "stepUpAuthAbove": "250.00"},
    "expires": "2026-06-30T23:59:59Z"
  }
}
```

A.4.3 Revocation model

- Use a status list or revocation registry referenced by VC status entries.
- Mirror the current status root or Merkle commitment on-chain for verifier auditability.
- Treat revocation as hard fail for payment and account opening unless an explicit emergency override policy exists.

A.5 Smart-contract and on-chain module design

Module	Purpose	Writes	Reads	Notes
AgentRegistry	Map Agent DID hash to current account, controller set hash, lifecycle status	create, suspend, deactivate, bindAccount	status for verifiers and wallet modules	Keep full DID doc off-chain; anchor hashes and pointers only

CredentialStatusAnchor	Anchor current credential-status root or list hash	updateRoot	status validation by verifier gateways	Lets off-chain status registries remain compact and flexible
PolicyRegistry	Store policy commitment hashes and active policy version for each agent	setPolicy, freezePolicy	wallet validator, auditors	Detailed policy stays off-chain; on-chain commitment prevents silent tampering
AgentAccountFactory	Deploy ERC-4337 accounts and bind them to agents	deployAccount, bindAgent	wallet discovery	Can enforce that only verified operators deploy bound accounts
PaymentReceiptRegistry	Store intent ID, receipt hash, execution status, and timestamp	recordReceipt, markReverted	audits and dispute review	Do not store sensitive purpose text or invoice data on-chain
EmergencyControl	Global or per-agent kill switch	pauseAgent, unpauseAgent	wallet validators, gateways	Must be tightly governed and highly auditable

A.6 ERC-4337 account design

- Validation logic should require: active Agent DID status, active policy commitment, unrevoked Authority VC reference, and acceptable execution mode for the requested action.
- Use session keys for short-lived automation. Session keys should be constrained by action type, expiry, asset, and spend caps.
- Use a paymaster only after preflight verification succeeds. This avoids subsidizing spam or malicious intent creation.
- Keep a separate governance/recovery key path from hot execution/session keys.
- For enterprise deployments, use treasury partitions or subaccounts so one agent compromise cannot drain a full treasury.

A.6.1 Recommended account modules

- PolicyValidator module: validates signed intent against current policy commitment and execution mode.
- SessionKey module: checks time-bounded delegated keys and their scope.
- SpendingLimit module: enforces rolling limits and nonce domains.
- CounterpartyGuard module: checks allowlists, deny lists, and sanctions/policy hooks.
- Recovery module: permits controlled rotation by governance keys after time lock or quorum.

A.7 Account opening flow

- 1. Owner completes KYC/KYB with a trusted issuer and receives or can request a KYC/KYB VC.
- 2. Platform creates Agent DID and Ownership VC linking agent to owner or institution.
- 3. Policy authority issues an Authority VC and stores a policy commitment in the PolicyRegistry.
- 4. AgentAccountFactory deploys an ERC-4337 smart account and binds it to the Agent DID.
- 5. Account Binding VC is issued; AgentRegistry status moves from pending to active.
- 6. Treasury funds the agent account or assigns a bounded budget partition.
- 7. Connectors and verifiers can now accept presentation of relevant credentials and route payment intents.

A.8 Payment intent and execution design

A.8.1 Suggested payment intent object fields

Field	Type	Description	Validation notes
intentId	bytes32 / UUID	Unique payment request identifier	Must be idempotent across retries
payerAgentDid	string	Agent DID of paying agent	Resolve and validate status before execution
payerAccount	address	Bound smart account address	Must match AgentRegistry binding
payeeRef	address or DID	Counterparty address or DID	DID form preferred where policy checks matter
asset	token identifier	Token or settlement asset	Must be allowed by policy
amount	decimal string / uint256	Requested transfer amount	Normalized before policy check
purposeCode	string enum	Reason for payment	Useful for reporting, policy, and dispute handling
connector	string enum	onchain, bridge, exchange, card, local rail	Connector-specific validators may apply
executionMode	enum	human_confirmed, pre_authorized, autonomous	Must not exceed authority VC scope

credentialRefs	array	Presented VC references or hashes	Verifier checks freshness and revocation
expiry	timestamp	Intent expiry	Expired intents hard fail
metadataHash	bytes32	Hash of rich off-chain context	Keeps chain data compact and private

A.8.2 Recommended execution states

- draft
- pending_verification
- awaiting_human_approval
- approved
- executing
- executed
- failed
- reverted
- expired

A.8.3 Payment execution sequence

- 1. Agent submits intent to the payment service with supporting credential references.
- 2. Verifier resolves DID, checks registry status, validates VCs and revocation state.
- 3. Policy engine computes decision: allow, hold for approval, or deny.
- 4. If needed, owner or approver signs an approval artifact bound to the same intent ID.
- 5. Smart account executes the transfer or connector call through ERC-4337 user operation or direct transaction.
- 6. Receipt hash and execution attestation are recorded on-chain; full logs and business context are stored off-chain.
- 7. Monitoring and limits are updated so future policy checks use fresh spend state.

A.9 Connector model

- On-chain token connector: direct stablecoin transfer or contract interaction.
- Bridge connector: moves value between on-chain agent funds and fiat or off-chain balances after KYC checks.
- Exchange connector: interacts with exchange subaccounts under additional venue credentials.
- Card or local-rail connector: only after a stronger identity, policy, and audit model is available; treat as phase-two scope.
- Each connector should expose the same minimal interface: quote, validate, execute, receipt, and reverse/reconcile where applicable.

A.10 Security model

- Threats to consider: stolen session key, forged credential presentation, policy bypass, replayed intent, malicious connector, compromised issuer, malicious owner, compromised paymaster, and spam intent flooding.
- Every intent should be nonced and idempotent.
- Policy checks should happen both preflight and at account validation time for high-value operations.
- Use short-lived session keys with purpose-specific scopes for autonomous mode.
- Freeze and revoke paths must be faster than ordinary execution paths.
- Keep comprehensive signed logs so post-incident attribution is possible.

Appendix B. Detailed implementation plan

This appendix converts the architecture into an execution program with milestones, workstreams, deliverables, and decision gates.

B.1 Workstreams

Workstream	Primary outputs	Dependencies	Lead profile
Identity and DID	DID method/profile, registries, issuer trust list, DID resolution service	governance and trust model	identity protocol engineer
Credentials and policy	VC schemas, issuance flows, status lists, policy language, decision engine	identity and compliance partners	applied cryptography / backend engineer
Wallet and account opening	ERC-4337 accounts, factory, recovery flow, budget partitions	policy commitments and DID binding	smart contract engineer
Payments	intent service, execution pipeline, receipt model, monitoring	wallet and policy engine	payments backend engineer
Compliance and ops	KYC/KYB process, audit logs, runbooks, emergency controls	credential schemas and revocation	compliance architect / security engineer

B.2 Twelve-week MVP plan

Weeks	Scope	Concrete deliverables	Exit criteria
1-2	Trust model and	Subject model, DID	Architecture approved;

	schemas	profile, three VC schemas, policy vocabulary, threat model v0	schemas frozen for MVP
3-4	Identity services	DID creation service, DID resolution service, issuer registry, status-list design	Agent DID and Owner DID can be created and resolved end-to-end
5-6	Credential issuance	OpenID4VCI issuance prototype, VC signing, status checks, presentation verifier	Ownership VC and Authority VC can be issued and validated
7-8	Wallet and binding	ERC-4337 account factory, AgentRegistry binding, recovery path, budget config	Bound account can be created for a verified agent
9-10	Payment intents	Intent schema, verification pipeline, policy engine MVP, human approval flow	Human-confirmed and pre-authorized payments succeed in sandbox
11-12	Hardening and audit	Receipt registry, freeze/revoke, observability, incident runbook, demo script	Demo shows create -> credential -> account -> intent -> execute -> audit trail

B.3 Six-month production roadmap

- Month 1-2: ship sandbox MVP with a single stablecoin on a single EVM L2 and one KYC/KYB issuer path.
- Month 3: add production-grade revocation, monitoring, paymaster controls, and enterprise treasury partitions.
- Month 4: add connector framework and one regulated bridge or exchange integration.
- Month 5: add autonomous mode with session-key restrictions and stronger approval thresholds.
- Month 6: begin partner pilots for agent-native account opening and bounded payment automation.

B.4 Team and capability plan

- Core protocol/identity engineer: DID profile, VC semantics, trust registry, verification logic.
- Smart contract engineer: ERC-4337 account modules, registries, receipt contracts, pause/recovery paths.
- Backend/platform engineer: issuance APIs, intent service, policy engine, observability, partner connectors.
- Security engineer: threat modeling, key management, secure signing, runtime monitoring, incident response.

- Compliance architect: KYC/KYB mapping, policy taxonomy, audit/export requirements, partner onboarding.

B.5 Product decisions to lock before coding

- Which DID profile is v1: did:web only, hybrid did:web plus chain anchor, or a bespoke did:agent method?
- Which credential formats are mandatory in v1: JSON-LD VC, JWT VC, or SD-JWT VC?
- Which chain and wallet stack is the reference environment for production pilots?
- Which autonomy modes are supported in v1: human-confirmed and pre-authorized only, or limited autonomous?
- What is the default settlement asset and what connectors are in scope for phase one?
- Which revocation path is authoritative during outages: off-chain status list, on-chain anchor, or both?

B.6 Success metrics

- Time to create and activate a verified agent account.
- Credential issuance success rate and verification latency.
- Payment intent approval latency by execution mode.
- Revocation propagation time from issuer action to effective denial.
- Rate of false-allow and false-deny policy outcomes.
- Number of incidents where freeze/recovery path worked within target SLA.

B.7 Recommended first pilot

- Use case: a treasury-owned software agent can pay approved service agents in USDC for bounded API or workflow execution.
- Why this is ideal: low-value, high-frequency, digitally verifiable, limited counterparties, and no need to solve full autonomous trading on day one.
- Pilot scope: one owner, one platform issuer, one EVM L2, one stablecoin, one or two approved counterparties, one KYC/KYB issuer, and one dashboard for approvals and freeze/recovery.

Appendix C. System architecture and component diagrams

This appendix adds concrete component boundaries, data flows, and deployment guidance so engineering, product, compliance, and security teams can align on a single implementation model.

C.1 Layered reference architecture

```
+-------------------------------------------------------------------------------
--+
|                                     Clients
|
|   Owner dashboard | Compliance console | Treasury console | Verifier/partner UI
|
+-------------------------------------------------------------------------------
--+
                                         |
                                         v
+-------------------------------------------------------------------------------
--+
|                               API / Gateway Layer
|
|  AuthN/AuthZ | Rate limits | Request signing | Audit correlation | Tenant routing
|
+-------------------------------------------------------------------------------
--+
                                         |
                 +------------------------+------------------------+
                 |                         |                        |
                 v                         v                        v
+---------------------------+ +---------------------------+ +---------------------
-+
| Identity & Credential     | | Policy / Decision Layer   | | Payment / Account
|
| Services                  | |                           | | Services
|
| - DID registry            | | - Policy compiler         | | - Account factory
|
| - DID resolver            | | - Rule engine             | | - Intent service
|
| - VC issuance             | | - Spend state tracker     | | - Execution router
|
| - VC presentation verify  | | - Approval workflow       | | - Connector manager
|
| - Revocation/status       | | - Risk scoring hooks      | | - Receipt service
|
+---------------------------+ +---------------------------+ +---------------------
-+
                 |                         |                        |
                 +------------------------+------------------------+
                                          |
                                          v
+-------------------------------------------------------------------------------
--+
|                         Chain / Smart Account / Registry Layer
|
```

```
| AgentRegistry | PolicyRegistry | ReceiptRegistry | AgentAccountFactory |
Paymaster  |
| Session keys | Spending limits | Account recovery | Connector adapters | Event
logs |
+-------------------------------------------------------------------------------
--+
                                          |
                 +------------------------+------------------------+
                 |                         |                        |
                 v                         v                        v
+---------------------------+ +---------------------------+ +---------------------
-+
| KYC / KYB issuer          | | Exchange / bridge / bank   | | Observability &
|
| credential provider       | | or payment-rail connectors | | security operations
|
|                           | |                            | |
|
+---------------------------+ +---------------------------+ +---------------------
-+
```

Implementation note: treat the identity services, policy services, and payment services as separate deployable domains even if a single team ships them initially. This keeps trust boundaries explicit and allows later substitution of issuers, policy authorities, or payment connectors without rewriting the whole stack.

C.2 Core component responsibilities

Component	Primary responsibility
AgentRegistry	Canonical registry for agent DID metadata, lifecycle state, wallet binding pointers, and suspension/freeze status.
Resolver Service	Resolves DID -> DID Document, retrieves revocation state, and normalizes controller + verification relationships.
Credential Service	Issues Ownership VC, KYC/KYB VC, Authority VC, Account Binding VC; stores issuance metadata and status list references.
Policy Engine	Evaluates payment intent against current credentials, policy commitment, connector constraints, and spend history.
Intent Service	Creates canonical payment intent IDs, idempotency keys, intent hashes, and approval challenges.
AgentAccountFactory	Deploys or registers ERC-4337 smart accounts and attaches policy/account modules.

Connector Manager	Normalizes quote/validate/execute/reconcile across on-chain token, exchange, and fiat bridge connectors.
Receipt Service	Produces machine-readable receipts, signed attestations, and on-chain receipt commitments.
Risk Monitor	Consumes events and alerts on anomalies: rapid spend bursts, repeated denials, connector failures, or policy bypass attempts.

C.3 Trust boundaries

- Issuer boundary: credential issuers are trusted only for the specific claim types they sign. A KYC issuer should not automatically be trusted as a policy authority.
- Execution boundary: payment execution services may prepare and route actions, but final execution authority should remain with the smart account validation layer and its policies.
- Observation boundary: analytics and monitoring systems can ingest event streams and redacted logs, but should not hold the authority to mutate account state.
- Recovery boundary: freeze, revoke, and rotate operations must use a separate control path from ordinary transaction submission.

Appendix D. Sequence diagrams

These text diagrams are intentionally implementation-oriented. They can later be converted into Mermaid, PlantUML, or BPMN with minimal changes.

D.1 Agent onboarding and account activation

```
Owner / Institution -> KYC Issuer: complete KYC/KYB workflow
KYC Issuer -> Owner Wallet: issue KYC/KYB VC
Platform -> Agent Registry: create Agent DID (pending)
Owner / Policy Authority -> Credential Service: request Ownership VC + Authority VC
Credential Service -> Agent Registry: store VC status refs + policy commitment
pointers
Platform -> AgentAccountFactory: deploy ERC-4337 smart account
Factory -> Agent Registry: bind account to Agent DID
Credential Service -> Owner / Agent Wallet: issue Account Binding VC
Agent Registry -> Platform: status = active
```

D.2 Human-confirmed payment

```
Agent -> Intent Service: create payment intent
Intent Service -> Policy Engine: evaluate intent
Policy Engine -> Credential / Resolver: verify DID + VC + revocation
Policy Engine -> Intent Service: decision = approval required
Intent Service -> Owner App: approval challenge
Owner App -> Intent Service: signed approval artifact
Intent Service -> Smart Account / Bundler: submit user operation
Smart Account -> Connector / Token: execute payment
Receipt Service -> Chain + Off-chain logs: store receipt hash + signed receipt
```

D.3 Pre-authorized autonomous payment

```
Agent -> Intent Service: create payment intent with session key proof
Intent Service -> Policy Engine: evaluate against bounded policy
Policy Engine -> Spend State: check rolling limit and counterparty rules
Policy Engine -> Smart Account / Bundler: allow execution
Smart Account -> Connector / Token: execute
Receipt Service -> Monitoring: emit success/failure and update spend state
```

D.4 Revocation and emergency freeze

```
Issuer / Governance -> Revocation Service: revoke VC or freeze agent
Revocation Service -> Status List / Chain anchor: publish new revocation state
Risk Monitor -> Policy Engine / Registry: mark agent high-risk or frozen
Any new intent -> Policy Engine: fetch latest state and deny execution
Recovery Authority -> Agent Registry / Account Module: rotate keys or unfreeze
after review
```

Appendix E. Smart contract interface draft

The interfaces below are intentionally minimal. They define module boundaries rather than the final production ABI. The goal is to keep identity, policy, and receipt concerns separate so implementations can evolve independently.

```
interface IAgentRegistry {
    struct AgentRecord {
        string agentDid;
        address smartAccount;
        bytes32 currentPolicyHash;
        bytes32 accountBindingVcHash;
        uint8 status; // 0 pending, 1 active, 2 suspended, 3 frozen, 4 revoked
        uint64 updatedAt;
    }

    event AgentRegistered(bytes32 indexed agentId, string agentDid, address indexed
account);
    event AgentStatusChanged(bytes32 indexed agentId, uint8 oldStatus, uint8
newStatus);
    event AgentPolicyUpdated(bytes32 indexed agentId, bytes32 oldPolicyHash,
bytes32 newPolicyHash);

    function registerAgent(bytes32 agentId, string calldata agentDid, address
smartAccount) external;
    function bindAccount(bytes32 agentId, address smartAccount, bytes32
accountBindingVcHash) external;
    function setPolicy(bytes32 agentId, bytes32 policyHash) external;
    function setStatus(bytes32 agentId, uint8 newStatus) external;
    function getAgent(bytes32 agentId) external view returns (AgentRecord memory);
}

interface IPolicyRegistry {
    event PolicyCommitted(bytes32 indexed policyHash, bytes metadataHash, uint64
effectiveFrom, uint64 expiresAt);
    event PolicyRevoked(bytes32 indexed policyHash, bytes32 reasonHash);

    function commitPolicy(bytes32 policyHash, bytes calldata metadataHash, uint64
effectiveFrom, uint64 expiresAt) external;
    function revokePolicy(bytes32 policyHash, bytes32 reasonHash) external;
    function isActive(bytes32 policyHash) external view returns (bool);
}

interface IReceiptRegistry {
    event ReceiptCommitted(bytes32 indexed intentId, bytes32 indexed receiptHash,
address indexed executor, uint64 timestamp);

    function commitReceipt(bytes32 intentId, bytes32 receiptHash) external;
    function receiptOf(bytes32 intentId) external view returns (bytes32);
}

interface IIntentExecutor {
    struct PaymentIntent {
        bytes32 intentId;
```

```
        bytes32 agentId;
        address payerAccount;
        address payee;
        address asset;
        uint256 amount;
        bytes32 policyHash;
        bytes32 purposeHash;
        uint64 deadline;
        uint8 executionMode; // 0 human_confirmed, 1 pre_authorized, 2 autonomous
        uint256 nonce;
    }

    event IntentValidated(bytes32 indexed intentId, bytes32 indexed agentId, uint8
decision);
    event IntentExecuted(bytes32 indexed intentId, address connector, bytes32
receiptHash);

    function validateIntent(PaymentIntent calldata intent, bytes calldata
authBundle) external view returns (bool ok, uint8 decision, bytes32 reasonHash);
    function executeIntent(PaymentIntent calldata intent, bytes calldata
authBundle, bytes calldata connectorData) external returns (bytes32 receiptHash);
}

interface IConnector {
    function quote(bytes calldata request) external view returns (bytes memory
response);
    function validate(bytes calldata request) external view returns (bool ok,
bytes32 reasonHash);
    function execute(bytes calldata request) external returns (bytes32
connectorReceiptHash);
    function reconcile(bytes32 connectorReceiptHash) external view returns (bytes
memory reconciliationData);
}
```

E.1 Contract design notes

- Keep registries thin. Store hashes and status on-chain; store large policy payloads, receipt payloads, and credential details off-chain with signed references.
- Split validation from execution. High-value flows should be re-validated at execution time, not only at API preflight time.
- Prefer module isolation. Spend limits, session keys, and recovery logic should be replaceable modules on top of a base smart account.
- Use explicit status enums and event logs. Compliance and incident response teams need deterministic audit trails.

Appendix F. API schemas and service contracts

The API layer should expose a stable machine-readable contract even if connector logic changes underneath. Below are suggested request/response shapes for the v1 platform.

F.1 Create Agent DID

```
POST /v1/agents
{
  "ownerDid": "did:web:org.example:treasury",
  "agentType": "payment-agent",
  "displayName": "treasury-usdc-agent-01",
  "controller": {"type": "service", "keyRef": "kms://tenant/keys/123"},
  "metadata": {"environment": "sandbox", "region": "us"}
}

201 Created
{
  "agentId": "agt_01J...",
  "agentDid": "did:agent:base:agt_01J...",
  "status": "pending"
}
```

F.2 Issue / attach credentials

```
POST /v1/credentials/authority
{
  "agentDid": "did:agent:base:agt_01J...",
  "issuerDid": "did:web:issuer.example",
  "grants": {
    "assets": ["USDC"],
    "maxPerTxn": "100000000",
    "dailyLimit": "500000000",
    "counterpartyMode": "allowlist",
    "allowlistedRecipients": ["0xabc...", "did:web:svc.example:billing-agent"],
    "executionModes": ["human_confirmed", "pre_authorized"]
  },
  "validUntil": "2027-12-31T00:00:00Z"
}
```

F.3 Open agent account

```
POST /v1/accounts
{
  "agentDid": "did:agent:base:agt_01J...",
  "chain": "base-mainnet",
  "accountTemplate": "erc4337-v1",
  "policyHash": "0x...",
  "initialBudget": {"asset": "USDC", "amount": "1000000000"}
}
```

F.4 Create payment intent

```
POST /v1/payment-intents
{
  "agentDid": "did:agent:base:agt_01J...",
  "payee": {"type": "address", "value": "0xdef..."},
  "asset": "USDC",
```

```
  "amount": "2500000",
  "purpose": "api_usage_invoice_2026_03",
  "executionMode": "pre_authorized",
  "connector": "onchain_token",
  "idempotencyKey": "pi-2026-03-16-001"
}

201 Created
{
  "intentId": "pi_01J...",
  "status": "pending_verification",
  "policyDecision": "allow",
  "expiresAt": "2026-03-16T12:00:00Z"
}
```

F.5 Approve intent

```
POST /v1/payment-intents/{intentId}/approve
{
  "approverDid": "did:web:org.example:treasury",
  "approvalSignature": "0x...",
  "note": "Approved under monthly budget"
}
```

F.6 Query receipt

```
GET /v1/payment-intents/{intentId}/receipt
200 OK
{
  "intentId": "pi_01J...",
  "status": "executed",
  "txHash": "0x...",
  "receiptHash": "0x...",
  "connector": "onchain_token",
  "executedAt": "2026-03-16T11:21:45Z"
}
```

F.7 Event contract

```
agent.created
agent.status.changed
credential.issued
credential.revoked
account.created
payment_intent.created
payment_intent.decisioned
payment_intent.approved
payment_intent.executed
payment_intent.failed
receipt.committed
risk.alert.raised
```

Appendix G. Risk matrix and control plan

This matrix is deliberately practical. It should be used as the baseline for security review, compliance sign-off, and pilot gating.

Risk	Likelihood	Impact	Primary controls
Credential forgery or misuse	Medium	High	Use issuer allowlists, signed credential verification, revocation checks, and short validity windows.
Stolen session key	Medium	High	Short-lived session keys, scope-bound grants, device/service attestation, rapid freeze path.
Policy bypass via connector edge case	Low to Medium	High	Re-validate at execution time, connector-specific policy adapters, invariant tests.
Replay of payment intent	Medium	Medium	Intent nonce domains, idempotency keys, deadlines, and duplicate receipt rejection.
Compromised issuer or policy authority	Low	Critical	Separate issuer roles, quorum controls, emergency trust-list updates, audit and rollback process.
Incorrect KYC mapping to wrong owner or agent	Low to Medium	High	Ownership VC review, issuer metadata checks, dual confirmation during activation.
On-chain/off-chain state divergence	Medium	High	Periodic reconciliation jobs, authoritative receipt hashes, status anchoring, repair tooling.
Abuse of autonomous	Medium	High	Default to human-

mode			confirmed or pre-authorized only, bounded scopes, anomaly detection, spend throttles.
Partner connector outage	High	Medium	Circuit breakers, quote fallback, retry logic, connector health scoring, rollback policies.
Privacy leakage from excessive on-chain metadata	Medium	High	Hash commitments only, minimal public fields, redaction of business context to off-chain storage.

G.1 Pilot gate checklist

- Only one chain, one stablecoin, and one or two approved counterparties in the first production pilot.
- Autonomous mode disabled until end-to-end revocation, freeze, and audit paths have been exercised in staging.
- At least one independent security review of account modules, registries, and intent validation logic before production funds are enabled.
- Issuers, connectors, and governance authorities documented in a machine-readable trust registry and a human-readable control matrix.

Appendix H. Business model and go-to-market chapter

The purpose of this chapter is to translate the technical stack into a business that can be piloted, sold, and expanded without over-promising. The recommended market position is not “another wallet” and not “a universal exchange.” The recommended position is: an identity-and-authority base layer for agent finance, with controlled account opening and programmable payment execution as the first paid products.

H.1 Product packaging

- Identity package: Agent DID creation, owner-agent binding, revocation, and trust registry access.
- Compliance package: issuance and verification of Ownership VC, KYC/KYB VC, and Authority VC with audit logs.
- Account package: ERC-4337 smart accounts, budget partitions, session keys, paymaster controls, and recovery paths.
- Payments package: payment intent API, policy evaluation, connector execution, receipts, and monitoring.

H.2 Target customers

- AI-native platforms that want their agents to hold budgets and pay for bounded services.
- Enterprise treasury or workflow teams that need software agents to execute limited payments under governance.
- Wallets, exchanges, or payment providers that want agent onboarding and policy-controlled execution without building identity from scratch.
- Agent marketplaces and service networks that need machine-readable identity, settlement, and receipts.

H.3 Monetization model

- Platform fee for verified agent account creation and lifecycle management.
- Credential issuance / verification fee per active agent or per credential set.
- Usage fee per payment intent evaluated or executed.
- Connector fee or revenue share on bridge / venue / regulated payment rails when applicable.
- Enterprise subscription for policy engine, trust registry, observability, and compliance exports.

H.4 Why the first wedge should be payments, not trading

- Payments have narrower policy surfaces and clearer approval semantics than fully autonomous trading.
- Payment receipts are easier to verify than trading profit attribution or best-execution claims.
- Enterprise and platform buyers can adopt bounded service payments before they are ready to authorize broader market activity.
- Once account, authority, and receipt primitives are trusted, the same base layer can be extended to venue access and trading rights.

H.5 Go-to-market sequence

- Phase 1: sandbox developer product for agent account creation and stablecoin payment intents.
- Phase 2: controlled pilot with one enterprise or platform treasury using approved counterparties only.
- Phase 3: add connector integrations and compliance reporting for regulated or hybrid settlement paths.
- Phase 4: expand to venue access credentials and narrowly scoped trading authority modules.

H.6 Positioning statement

- The company should position itself as the trust, authority, and execution control layer for agent finance.
- It should avoid positioning itself as a broad consumer wallet or a full autonomous trading venue in v1.

Appendix D. AP2 compatibility and adoption plan

This appendix turns the AP2 review into an implementation decision. The conclusion is to own the identity-and-authority substrate, then expose a compatibility layer that can emit or consume AP2-style mandate artifacts where commerce and payment integrations require them.

D.1 What AP2 already provides well

- A role-based model for user, shopping agent, merchant, credential provider, issuer, and payment processor interactions.
- A mandate-centered trust model built around Cart Mandate, Intent Mandate, and Payment Mandate.
- Human-present and human-not-present transaction journeys, including challenge escalation to a trusted surface.
- Dispute and liability framing that preserves evidence about user authorization, merchant commitment, and payment execution context.
- A practical bridge into existing payment ecosystems that still expect merchant, issuer, and challenge semantics.

D.2 Where AP2 stops and our stack begins

AP2 is not a full on-chain identity, account, and policy substrate. It does not give a complete design for agent DID lifecycle, owner–agent binding, KYC/KYB credential schemas, smart-account opening, revocation registries, budget partitions, or generalized agent-to-agent financial authority. Those are the layers we must own to support not only commerce checkout, but also treasury, wallet, cross-venue access, and later agent-to-agent payments and trading.

D.3 Recommended layering

- Layer 1 — Identity: Owner DID, Institution DID, Agent DID, DID resolution, controller changes, key rotation, suspension, and deactivation.
- Layer 2 — Credentials: Ownership VC, KYC/KYB Result VC, Authority VC, Status/Risk VC, plus presentation and revocation checks.
- Layer 3 — Accounts: ERC-4337 or equivalent smart accounts, treasury partitions, session keys, signer management, and account status registry.
- Layer 4 — Policy: amount limits, asset scope, counterparty allowlists, venue scope, geography scope, challenge thresholds, and kill-switches.
- Layer 5 — Execution: payment intent evaluation, stablecoin transfer or connector call, receipt anchoring, and audit logging.
- Layer 6 — AP2 compatibility: transform internal intents and authority proofs into Cart/Intent/Payment-mandate style payloads for external flows.

D.4 Mapping our objects to AP2 concepts

Internal object mapping should be explicit. Ownership VC maps to the trusted proof that a given agent is acting for a responsible owner. Authority VC plus policy evaluation maps to the precondition for any AP2-like Intent Mandate. A user-confirmed checkout object can be projected into a Cart-Mandate-like artifact. A payment-intent execution approval can be projected into a Payment-Mandate-like artifact suitable for merchant, processor, issuer, or credential-provider flows. This means AP2 becomes a serialization and interop target for delegated payment proof, not the source of truth for identity or permissions.

D.5 Detailed build plan

Phase 0 — protocol definition

- Choose DID strategy: hybrid recommendation is web-backed issuer and verifier trust domains with on-chain anchoring for agent/account state.
- Define core schemas: Ownership VC, KYC/KYB Result VC, Authority VC, Status/Risk VC, Payment Intent, Receipt Attestation, Challenge Record.
- Define revocation model: credential status list or status endpoint, plus on-chain account/agent suspension flags.
- Define policy semantics and execution modes: human-confirmed, pre-authorized, and fully agent-native.

Phase 1 — identity and account substrate

- Implement DID issuance and resolution for owner, institution, and agent subjects.
- Implement ownership binding and key-rotation flows.
- Integrate KYC/KYB provider and issue result credentials rather than storing raw KYC payloads in the product domain.
- Create agent smart accounts and treasury partitions after credential validation.
- Implement account registry contract with fields for agent DID, owner DID reference, account address, policy hash, status, and timestamps.

Phase 2 — policy and payment execution

- Implement payment-intent API that captures payer DID, payee, asset, amount, purpose, authority reference, and execution mode.
- Implement policy engine that resolves DIDs, verifies credential validity, checks revocation, evaluates allow/hold/deny, and determines whether a trusted-surface challenge is required.
- Support stablecoin payment execution first; treat card or fiat connectors as later adapters.
- Record receipt hashes and execution attestations on-chain while storing rich logs off-chain.

Phase 3 — AP2 interop adapter

- Implement transformation logic from internal authority and checkout objects into AP2-style Cart Mandate, Intent Mandate, and Payment Mandate payloads when an external partner expects them.
- Implement trusted-surface challenge routing compatible with AP2-style flows for higher-risk actions.
- Implement dispute package generation so the system can export user authorization evidence, merchant commitment evidence, and payment execution evidence in AP2-friendly form.

D.6 Trusted surface model

A trusted surface is mandatory for higher-risk approvals. In our architecture the trusted surface is not the agent runtime itself. It is a user-controlled or institution-controlled channel, such as a wallet app, banking app, custody console, or enterprise approval surface. The policy engine should be able to require trusted-surface confirmation based on amount, counterparty, geography, asset type, or anomaly score. This directly aligns with AP2's challenge and step-up logic while keeping our internal identity and policy substrate chain-native.

D.7 Technical design choices we should lock early

- Whether the primary trust anchor is did:web plus on-chain anchoring, or a fully on-chain DID method from day one.
- Whether credentials are held directly by the owner wallet, by a custody service, or by a credential vault bound to the smart account.
- Whether the policy engine runs fully off-chain with on-chain commitments, or partially on-chain for a subset of hard rules.
- Whether AP2 compatibility is emitted only for merchant/checkout connectors, or generalized to all delegated payment authorizations.
- Whether challenge fulfillment returns a fresh short-lived approval credential, a signed attestation, or a one-time session capability.

D.8 Why this strategy is preferable

This strategy preserves control over the most defensible layers: identity, account opening, authority, policy, and receipts. It avoids locking the stack to a commerce-specific protocol while still making the platform immediately interoperable with an important emerging standard for agent payments. In short: we build the substrate, then speak AP2 at the edge.

D.9 Additional AP2 references

AP2 specification — https://ap2-protocol.org/specification/

Life of a Transaction — https://ap2-protocol.org/topics/life-of-a-transaction/

Google Cloud AP2 announcement — https://cloud.google.com/blog/products/ai-machine-learning/announcing-agents-to-payments-ap2-protocol

Google Agentic Commerce AP2 repository — https://github.com/google-agentic-commerce/AP2

DeepWiki AP2 overview — https://deepwiki.com/google-agentic-commerce/AP2

E. Token economy implications for Web4 agent financial infrastructure

This section integrates the recent "token economy" framing from NVIDIA GTC into the architecture. The goal is not to introduce a speculative crypto-token thesis. Instead, the point is that inference tokens and related AI-compute quotas are emerging as first-class economic units for agent systems. In a Web4 market, agents may need to manage not only money, but also compute budget, action budget, and policy-constrained access to tools and models.

E.1 Why token economy matters to this architecture

The current white paper already argues that agent finance should begin with identity, authority, account opening, and payment. The token-economy lens extends that thesis: the account of an autonomous agent should not be treated as a simple crypto wallet. It should be treated as a multi-resource operating account that can hold and govern several classes of value at once.

- Monetary value: stablecoins, crypto assets, settlement balances, and withdrawals.
- Compute value: inference-token quotas, model-usage credits, tool-call quotas, and API budgets.
- Authority value: policy-based rights to spend, trade, route, invoke tools, or escalate to challenge flows.
- Reputation value: trust score, execution reliability, policy history, and revocation status.

The practical implication is that Web4 infrastructure should manage money, compute, and authority together. If future agents consume and exchange compute-intensive services, then many economically meaningful transactions will begin as resource-allocation events and only later settle as money movements.

E.2 Expanded account model: from wallet to multi-resource treasury account

A conventional wallet model is too narrow. The recommended design is an Agent Treasury OS that maintains a programmable operating account for each agent. That account can contain multiple ledgers or partitions:

- Cash ledger: stablecoin balances and settlement assets.
- Compute ledger: inference-token quotas, model-usage allowances, or prepaid provider credits.
- Tool ledger: service-specific credits for retrieval, data feeds, verification providers, or external APIs.
- Risk ledger: maximum daily loss, maximum position size, challenge thresholds, and escalation flags.
- Authority ledger: active mandates, delegated permissions, venue access rights, and revocation state.

This turns the agent account into a financial-and-operational control plane. It is no longer just an address that signs. It becomes the place where budgeting, execution rights, and settlement capacity are jointly managed.

E.3 New credential types required by a token-economy model

The original credential set—Ownership VC, KYC/KYB VC, Authority VC, and Policy VC—should be extended to cover resource economics. Suggested additions include:

- Compute Budget Credential: states how many inference tokens, provider credits, or model-usage units an agent may consume in a period.
- Provider Access Credential: states which model providers, tool providers, or execution networks the agent may use.
- Conversion Credential: states whether and how compute budget can be converted into monetary settlement or vice versa.

- Replenishment Credential: states whether the system may auto-top-up compute or payment budget under pre-agreed policy limits.
- Execution Class Credential: states whether the agent can perform advice-only, user-confirmed, pre-authorized, or fully agent-native actions in a given domain.

These credentials should still be expressed as verifiable credentials rather than packed directly into the DID Document. The DID remains the subject identifier; the credentials express rights, status, and limits.

E.4 Resource intents as a sibling to payment intents

The existing architecture introduces Payment Intent objects to separate "requesting a financial action" from "executing a transfer." The token-economy extension should add a sibling object: Resource Intent. This represents a request to allocate, purchase, consume, convert, or replenish non-cash resources such as inference tokens or tool credits.

Suggested Resource Intent fields:

- requestorAgentDid
- ownerDid or institutionDid
- resourceType (inference, model-credit, tool-credit, data-credit, bandwidth, etc.)
- providerId or providerDid
- quantity and unit
- maxSpend and settlementAsset
- purpose and workflowRef
- policyRef and authorityRef
- executionMode (human-confirmed, pre-authorized, fully agent-native)
- expiry, nonce, and replay protections

A Resource Intent may later settle as money, but the object itself should express the economics in native resource units. This avoids forcing every action into a stablecoin-denominated abstraction too early.

E.5 Settlement patterns for token economy

The token-economy view changes settlement design. Not every economically meaningful event should trigger an immediate on-chain transfer. The stack should support three settlement patterns in parallel:

1. Real-time settlement. Best for large transfers, high-value agent-to-agent actions, or low-trust counterparties. The payment or resource purchase settles immediately on-chain.

2. Batched settlement. Best for frequent micro-usage, such as repeated agent API calls or model invocations. Resource usage is metered continuously and netted into periodic settlement.

3. Internal-ledger settlement. Best for intra-platform and intra-enterprise workflows. Resource balances and monetary balances are updated on an internal ledger, with periodic reconciliation and optional on-chain anchoring.

This design allows the platform to support high-frequency machine activity without making every tiny event an expensive settlement transaction.

E.6 How token economy changes the Web4 infrastructure map

The architecture should therefore be understood as a stack with five tightly coupled cores:

- Agent Passport: identity, ownership, KYC/KYB linkage, and reputation.
- Agent Mandate Network: delegated authority, challenge rules, trusted surfaces, and execution boundaries.
- Agent Treasury OS: smart accounts, sub-accounts, budget ledgers, and programmable controls.
- Agent Settlement Rail: money settlement, resource settlement, batching, escrow, and receipts.
- Agent Venue Access Layer: trade intents, resource-provider adapters, payment connectors, and market access adapters.

The token-economy view strengthens the case that the real opportunity is not a single wallet or payment app. The opportunity is the control substrate that makes money, compute, and authority interoperable.

E.7 Technical components we should add to the roadmap

If the token-economy model is adopted, the implementation roadmap should add the following components:

- Compute Budget Registry: tracks available inference and tool budgets by agent and owner.
- Provider Quota Connectors: adapters to model providers, tool providers, or marketplaces that expose metered usage and balance state.
- Resource Metering Service: normalizes usage events into a canonical format for audit, pricing, and settlement.
- Budget-to-Money Converter: translates usage into money obligations under signed pricing schedules.
- Auto-Replenishment Policy Engine: allows tightly scoped top-ups based on approved thresholds.
- Unified Receipt Model: links resource consumption, payment authorization, and settlement outcomes into one audit object.

E.8 Suggested product positioning

With token economy integrated, the product should not be positioned merely as an on-chain payment substrate. A stronger position is: “an identity, authority, treasury, and settlement substrate for agent-native financial and compute markets.” That phrase better captures that the platform is designed for both monetary transactions and machine-native resource economics.

E.9 What this changes for go-to-market

The first realistic commercialization opportunities become clearer when token economy is included:

- Agent API and tool-call settlement for AI-native products.
- Enterprise treasury controls for multi-agent software systems.
- Compute-budget passports for organizations that want to govern model access and spending.
- Cross-provider settlement for AI workflows that span multiple model and tool vendors.
- Payment-plus-compute reporting for regulated or audited environments.

This widens the product beyond “agent payments” and makes it useful to AI engineering, enterprise controls, and financial infrastructure buyers.

E.10 Bottom line

The token-economy perspective does not replace the original identity-first architecture. It sharpens it. It suggests that the future agent account must manage more than financial balances: it must also manage

compute rights, action rights, and conversion paths between resources and money. For this reason, DID, VC, account opening, policy, and settlement remain the correct starting point; they simply need to be expanded into a multi-resource framework rather than a money-only framework.

F. OpenAI super-app and Google WebMCP implications for Web4 agent financial infrastructure

This section integrates two recent external signals into the white paper. First, OpenAI's reported effort to combine the ChatGPT app, browser, and Codex into a desktop "super app" indicates a strategic move to own the primary user and agent entry point for context, tools, and action execution. Second, Google's WebMCP work indicates a push to make websites expose structured, agent-ready capabilities instead of forcing browser agents to rely on brittle DOM manipulation. Together, these signals strengthen the case that the most valuable Web4 financial opportunity is not a single wallet or payment button, but a unified financial action layer that agents can call natively.

F.1 Why these two signals matter

The OpenAI signal matters because it shows that the next platform battle may be about control of the default action interface: where user intent, context, tool access, and task execution come together. If a super-assistant becomes the primary operating interface, the financial system behind it must be callable as structured actions, not just exposed as human-facing screens.

The Google WebMCP signal matters because it suggests that the web itself is evolving from "pages that humans click" into "capability surfaces that agents can invoke." If websites are beginning to expose structured tools to agents, then finance should do the same: account opening, identity presentation, budget checks, payment authorization, settlement, and venue access should all be represented as explicit, typed actions.

F.2 Strategic implication for our architecture

The existing white paper already argues for DID, VC, account, policy, payment, and settlement as the core substrate. The OpenAI and Google developments sharpen that argument and add a missing architectural layer: a capability-exposure layer for agent-native financial actions.

The revised stack should therefore be understood as follows:

- Identity layer: Owner DID, Institution DID, Agent DID, and lifecycle state.
- Credential layer: ownership, KYC/KYB, authority, policy, compute budget, and provider access credentials.
- Account layer: smart accounts, treasury sub-accounts, and budget ledgers.
- Policy and mandate layer: AP2-inspired mandates, challenge rules, trusted surfaces, and execution boundaries.
- Financial capability exposure layer: structured declarative and imperative actions that agents can call directly.
- Settlement and venue layer: payment intents, resource intents, stablecoin settlement, connectors, and trade/venue adapters.

F.3 New layer: financial capability exposure

This new layer is the direct design response to WebMCP. Instead of requiring agents to drive a wallet UI, browser extension, or exchange front end, the platform should expose machine-readable financial capabilities. In other words, our system should become "agent-ready" in the same sense that WebMCP proposes websites should become agent-ready.

Two action classes are recommended:

- Declarative financial actions: highly structured, bounded actions that can be expressed with stable schemas and simple validation. Examples include resolveIdentity, presentCredential, createAccount, getBudget, createPaymentIntent, authorizePayment, getReceipt, and getSettlementStatus.
- Imperative financial actions: more dynamic, multi-step, or context-sensitive actions that require additional planning or execution logic. Examples include routePayment, rebalanceTreasury, performMultiVenueQuote, submitTradeIntent, settleEscrow, and convertComputeBudgetToStablecoin.

This split mirrors Google's declarative and imperative WebMCP model and gives the protocol a clean way to support both safe, routine operations and more complex workflows.

F.4 OpenAI-style entry point implications

If OpenAI succeeds in making a unified desktop super app the default interface for work and agentic action, then finance should not remain fragmented across separate KYC portals, wallet screens, payment pop-ups, and exchange UIs. Instead, the financial substrate should behave like a single action environment.

For our design, this means:

- Intent-first workflows should be the norm. A user or agent should express a financial objective, and the system should translate it into typed intents and authorized execution paths.
- Context should travel with the action. Identity state, budget state, authority state, risk state, and prior receipts should be available to the orchestration layer.
- The primary object is not a page or form, but an action. This strongly supports the white paper's use of Payment Intent, Resource Intent, and, later, Trade Intent objects.

F.5 What this means for product strategy

OpenAI's super-app direction suggests that the winning layer may be the default action entry point. Google's WebMCP suggests that the winning protocol may be the one that standardizes structured capability access. Together, they imply that our positioning should evolve from "agent payment substrate" toward "agent financial action layer."

A stronger positioning statement is:

> ***An identity, authority, account, and settlement substrate that exposes agent-native financial actions for Web4.***

F.6 Concrete capability set to add to the roadmap

The following capability registry should be added to the roadmap and exposed through SDK/API/tool interfaces:

- resolveIdentity

- presentCredential
- createAgentAccount
- getAccountState
- getBudgetState
- createPaymentIntent
- authorizeIntent
- executePayment
- createResourceIntent
- authorizeResourceSpend
- settleUsage
- requestQuote
- submitTradeIntent
- freezeAgent
- revokeCredential
- getReceipt

These functions are not just APIs; they are the candidate building blocks of a future Web4 financial capability protocol.

F.7 Build recommendation

We should not try to build a browser or a chat super app. We should build the financial execution substrate that a future super app, agent runtime, or browser-agent environment can call. That means:

- Keep DID, VC, account, and policy as the core proprietary layer.
- Remain AP2-compatible for mandates, challenge, trusted surface, and payment authorization semantics.
- Add a capability-exposure layer inspired by WebMCP so that finance is callable as structured actions.
- Treat the unified entry-point trend as validation that control of action execution matters more than control of isolated UI screens.

F.8 Suggested white-paper conclusion after these updates

Taken together, the token-economy framing, AP2-compatible authorization logic, OpenAI's move toward a unified super-assistant entry point, and Google's move toward agent-ready web capabilities all point in the same direction: Web4 financial infrastructure should be built as a structured action substrate. Its foundation is identity and authority; its operating core is programmable account and treasury control; its execution model is intent-, mandate-, and policy-driven; and its external interface is a standard set of agent-native financial capabilities.